

Customer Support Triage System

1. Objective / Goal

Build an LLM-powered system that automatically classifies incoming customer support queries into predefined categories and assigns a priority level.

Why are we building this?

Customer support teams deal with large volumes of unstructured queries. Manual triage leads to:

- Incorrect routing
- Delayed response times
- Increased SLA breaches

This system ensures **instant, consistent, and scalable triaging** of tickets.

2. Problem Statement

Customer queries arrive in free-text format without structured metadata. Support agents manually interpret and route them, which leads to:

- Misclassification (e.g., billing vs complaint confusion)
- Delayed escalation of urgent issues
- Increased operational load

We need an automated system that:

- Understands intent from natural language
- Assigns category + priority reliably
- Provides reasoning for transparency

3. Target Users

- Customer Support Teams
- Support Ops Managers
- Early-stage startups scaling support
- Enterprises handling high ticket volumes

4. Functional Requirements

4.1 Input Handling

- Accept raw text input (customer query)
- Support single query at a time (batch optional stretch)

4.2 Classification

The system must classify queries into **one of the following categories**:

- Billing
- Technical Issue
- Account Issue
- Complaint

4.3 Priority Assignment

Assign one of:

- Low
- Medium
- High

Based on:

- Urgency indicators (e.g., "urgent", "ASAP")
- Severity (financial loss, service disruption)
- Emotional tone

4.4 Reason Generation

System must generate a short explanation:

- Why a category was chosen
- Why a priority was assigned

4.5 Output Format

Strict JSON output:

```
{  
  "category": "<category>",  
  "priority": "<priority>",  
  "reason": "<short explanation>"
```

```
}
```

5. System Design (LLM Pipeline)

5.1 Core Components

1. **Input Layer**
 - Text input (CLI / simple UI / notebook)
2. **Context Builder - Template Layer**
 - Injects user query into template
 - Zero-shot or few-shot prompting
3. **LLM Layer**
 - OpenAI / Gemini / local LLM
4. **Output Parser**
 - Ensures strict JSON structure
 - Handles malformed outputs

5.2 Pipeline Flow

User Query → Prompt Template → LLM → JSON Output → Parsed Response

6. Sample Inputs & Outputs

Example 1

Input:

"I was charged twice for my subscription and no one is responding. This is urgent!"

Output:

```
{
  "category": "billing",
  "priority": "high",
  "reason": "double charge and urgency mentioned"
}
```

Example 2

Input:

"My app keeps crashing when I try to upload files."

Output:

```
{  
  "category": "technical issue",  
  "priority": "medium",  
  "reason": "application malfunction without urgency"  
}
```

Example 3

Input:

"I want to change my email address linked to my account."

Output:

```
{  
  "category": "account issue",  
  "priority": "low",  
  "reason": "routine account modification request"  
}
```

7. Implementation Plan

Phase 1: Setup

- Choose LLM (OpenAI / Gemini)
- Setup API key / SDK
- Basic Python script

Phase 2: Prompting

- Implement prompt template
- Add few-shot examples (optional boost)

Phase 3: Output Structuring

- Parse response into JSON
- Add fallback handling:
 - Retry on invalid JSON
 - Regex extraction (if needed)

Phase 4: Interface

- CLI OR Streamlit UI:
 - Input box
 - Output display

8. Technical Stack

- Language: Python
- LLM: OpenAI GPT / Gemini
- Interface: CLI or Streamlit
- Parsing: `json`, `pydantic` (optional)

9. Deliverables

- Python script / notebook
- Prompt template
- Sample test cases
- JSON output validation logic
- (Optional) Simple UI

10. Future Enhancements (Not in Current Scope)

- Multi-label classification
- Ticket auto-routing integration (Zendesk, Freshdesk)
- Sentiment analysis layer
- Multi-language support
- Fine-tuned classification model

11. Key Value Proposition

- Instant ticket triaging
- Reduced manual workload
- Faster resolution cycles
- Improved customer satisfaction